

# claude-windows / d596f7ef-c2b1-4b96-b022-918ac3e40023

## Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\d596f7ef-c2b1-4b96-b022-918ac3e40023.jsonl`
- SHA-256: `29c6539f57893682054e98c7ce54f1428757915ce976d35675f8d0e52ffda57c`
- Source modified: `2026-07-09T04:49:14+00:00`
- Imported at: `2026-07-09T08:30:26+00:00`
- project: `C--Users-avidu-Projects-badminton-highlight-indexer`
- session_id: `d596f7ef-c2b1-4b96-b022-918ac3e40023`
```

## Transcript

### 1. user (2026-07-09T04:23:22.804Z)

Two tests fail on local Windows (they pass on CI Linux, so master's CI is green and this is a latent portability bug from the #535 GCSFuse work):

```
- tests/test_storage.py::test_gcs_fuse_helpers
- tests/test_storage.py::test_acquire_local_input_uses_gcs_fuse_when_available
```

Failure (test\_gcs\_fuse\_helpers, ~tests/test\_storage.py:579):

```
assert fused == str(mount / "videos/clip.mp4")
AssertionError: 'C:\\Users\\...\\videos\\clip.mp4' == 'C:\\Users\\...\\videos\\clip.mp4'
```

Root cause: the GCSFuse path helper in backend/storage/\_\_init\_\_.py (the \_gcs\_mount\_root / gcs-fuse path join added in #535 ? `perf(compile): use GCSFuse mount for source in worker to avoid full re-download`) builds the mounted path with a forward slash ("videos/clip.mp4"), while the test compares against str(Path(mount) / "videos/clip.mp4") which is OS-native (backslash on Windows). On Linux both are "/", so CI passes; on Windows they diverge.

Decide the correct contract and fix consistently: a GCSFuse mount path is a real local filesystem path, so it should almost certainly be OS-native (use os.path.join / Path, not hardcoded "/"), OR ? if the code intentionally keeps POSIX separators because the mount only ever exists on the Linux worker ? make the TEST assert the POSIX form and mark it linux-only. Then run `python run\_all\_tests.py --cov=backend --cov=training --cov-fail-under=70` on Windows and confirm both tests pass (full suite currently 1933 passed / 2 failed / coverage 84.28%).

Context: surfaced while running the pre-commit gate for PR #547 (scripts/gates/ perf gate + nightly reorg), which does not touch backend/storage ? this is unrelated and pre-existing.

### 2. assistant (2026-07-09T04:23:30.618Z)

I'll start by locating the actual code, since memory notes the primary working dir may be a stub. Let me read the relevant files in parallel.

### 3. user (2026-07-09T04:23:31.686Z)

<system-reminder>This memory is 2 days old. Memories are point-in-time observations, not live state ? claims about code behavior or file:line citations may be outdated. Verify against current code before asserting as fact.</system-reminder>

```
1    ---
2    name: actual-repo-location
3    description: "The badminton-highlight-indexer code lives under khelsutra-guru/, not the
primary working dir stub"
4    metadata:
5        node_type: memory
6        type: project
7        originSessionId: df5d5fb2-534a-447a-8181-b222f94655f3
8    ---
```

9  
10 The primary working directory `C:\Users\avidu\Projects\badminton-highlight-indexer` is a  
near-empty  
11 **\*\*stub\*\*** ? it contains only `AGENTS.md`, an empty `.claude/`, and  
`output/nightly\_regression/`  
12 (a `LATEST\_STATUS.txt`). The actual `khelsutra-guru/badminton-highlight-indexer`  
codebase lives at:  
13  
14 `C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer`  
15  
16 (git remote `github.com/Khelsutra/badminton-highlight-indexer`, default branch  
`master`).  
17  
18 **\*\*How to apply:\*\*** when a task references this repo's code (backend/worker,  
pipeline/segmenters,  
19 config.json, run\_all\_tests.py, etc.), `cd` to the khelsutra-guru path and work there ?  
the stub dir  
20 has none of it. Confirmed 2026-07-06 while fixing the wf294 silent-detector-failure  
issue.  
21  
22 Repo conventions verified there: CI = ruff 0.15.16 (`ruff check .`), mypy 2.1.0 (`mypy  
backend  
23 training`; `mypy.ini` quarantines several modules incl. `backend.worker.\_\_main\_\_`,  
`gemini`,  
24 `yolo\_hybrid` via `ignore\_errors=True`), full suite via `python run\_all\_tests.py`,  
coverage floor  
25 `--cov-fail-under=70` (actual ~86%). Segmenters use a `Success`/`Failure` Result  
contract  
26 ([[efficient-timeouts-and-retries]] is unrelated).  
27

#### 4. user (2026-07-09T04:23:32.293Z)

No files found

#### 5. user (2026-07-09T04:23:32.349Z)

No files found

#### 6. assistant (2026-07-09T04:23:38.867Z)

The code lives under `khelsutra-guru/`. Let me find the actual files there.

#### 7. user (2026-07-09T04:23:39.991Z)

C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\backend\storage\\_\_init\_\_.py

#### 8. user (2026-07-09T04:23:40.242Z)

C:\Users\avidu\Projects\khelsutra-guru\badminton-highlight-indexer\tests\test\_storage.py

#### 9. assistant (2026-07-09T04:23:46.136Z)

Both exist. Per the repo-freshness convention, let me check git state and pull before reading,  
then read the files.

#### 10. user (2026-07-09T04:23:48.512Z)

?? "SESSION\_HANDOFF (1).md"  
---BRANCH---  
codex/issue-523-sd-reel-plan-clear  
---LOG---  
a460d74 chore(handoff): close issue #523 and move to next open bugs  
b3e1b4e chore(handoff): refresh session state for sd reel follow-up

ad9c5af feat(compile): expose rendition selector in legacy UI

## 11. user (2026-07-09T04:23:48.551Z)

```
1      """Pluggable storage layer: local (default) + S3/GCS first-class buckets + Google Drive.
2
3      URI-driven (see ``ref.parse_uri``). The package import is cheap ? cloud SDKs are
imported
4      lazily, only when a non-local bucket URI is actually used (install the ``storage-s3`` /
5      ``storage-gcs`` extras). The default ``local`` backend needs no third-party deps and
preserves
6      today's behaviour byte-for-byte; ``gdrive://`` likewise needs none ? it reads a locally
MOUNTED
7      Google Drive (Google Drive for Desktop) as plain files.
8
9      Worker/dispatcher entry points speak URIs via the thin helpers ``fetch`` / ``put`` /
10     ``list_uris`` / ``get_storage``; ``local://`` (and bare paths) make ``fetch`` an
identity
11     passthrough. See ``docs/DECOUPLED_COMPUTE``.
12     """
13
14     from __future__ import annotations
15
16     import os
17     import shutil
18     import tempfile
19     from typing import Any, Dict, Iterator, Optional
20
21     from backend.storage.base import StorageBackend
22     from backend.storage.ref import StorageRef, StorageStat, parse_uri, resolve_input_uri
23
24     __all__ = [
25         "StorageBackend",
26         "StorageRef",
27         "StorageStat",
28         "parse_uri",
29         "resolve_input_uri",
30         "get_storage",
31         "fetch",
32         "put",
33         "exists",
34         "signed_url",
35         "list_uris",
36         "open_writer",
37         "acquire_local_input",
38         "cleanup_acquired_local_input",
39         "input_is_local",
40         "resolve_input_ref",
41     ]
42
43
44     def _storage_dict(storage_config: Any) -> Dict[str, Any]:
45         """Coerce a StorageConfig (Pydantic) / dict / None into a plain settings dict."""
46         if storage_config is None:
47             return {}
48         if hasattr(storage_config, "model_dump"):
49             return storage_config.model_dump() # type: ignore[no-any-return]
50         return dict(storage_config)
51
52
53     def resolve_input_ref(raw: str, storage_config: Any = None) -> StorageRef:
54         """Resolve a user-supplied input (a path or storage URI) into a ``StorageRef`` under
the input
55         config (``input_backend``/``input_root``). Raises ``ValueError`` for an unsupported
URI scheme ?
```

```

56     callers at the API/CLI boundary catch that and return a clean client error (never a
57     500).
58     Use ``ref.backend`` to branch local-vs-remote and ``ref.key`` for the resolved LOCAL
59     path (e.g. ``local:///`` is stripped to the bare path), rather than stat-ing the raw URI
60     string."""
61     sc = _storage_dict(storage_config)
62     uri = resolve_input_uri(
63         raw,
64         input_backend=sc.get("input_backend", "local"),
65         input_root=sc.get("input_root", ""),
66     )
67     return parse_uri(uri)
68
69     def input_is_local(raw: str, storage_config: Any = None) -> bool:
70         """True if ``raw`` resolves (under the input config) to a LOCAL filesystem path ?
71         i.e. a plain on-disk file the caller can stat/validate directly. False for a bucket / Drive
72         URI that must be fetched first. Raises ``ValueError`` for an unsupported scheme (same as
73         ``resolve_input_ref``)."""
74         return resolve_input_ref(raw, storage_config).backend == "local"
75
76     def acquire_local_input(
77         raw: str, storage_config: Any = None, *, scratch_parent: "str | None" = None
78     ) -> str:
79         """Resolve ``raw`` (a path or storage URI) to a LOCAL file path ready for
80         processing.
81         Local / bare paths are returned UNCHANGED (identity passthrough, no copy ? today's
82         behaviour byte-for-byte). A bucket / Drive source (``gs:///``/``s3:///``/``gdrive:///``, or a
83         bare name under a non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir
84         and the local path returned. The caller owns successful remote scratch copies; use
85         ``cleanup_acquired_local_input`` when the consumer has finished reading the returned
86         path."""
87         sc = _storage_dict(storage_config)
88         ref = resolve_input_ref(raw, sc)
89         if ref.backend == "local":
90             return (
91                 ref.key
92             ) # identity ? no temp, no copy, byte-identical to passing the path directly
93         scratch = tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
94         try:
95             return fetch(ref.uri, os.path.join(scratch, ref.name or "in.mp4"), config=sc)
96         except Exception:
97             shutil.rmtree(scratch, ignore_errors=True)
98             raise
99
100     def cleanup_acquired_local_input(
101         raw: str, local_path: str, storage_config: Any = None
102     ) -> None:
103         """Delete a remote scratch dir produced by ``acquire_local_input``.
104         Local inputs are identity paths and must never be removed. Remote acquisitions are
105         downloaded into a fresh ``rally-input-`` directory; deleting that directory is safe once the
106         caller has finished hashing/validating/stitching/segmenting the returned file.

```

```

109     """
110     try:
111         if input_is_local(raw, storage_config):
112             return
113     except ValueError:
114         return
115     if not local_path:
116         return
117     scratch = os.path.dirname(os.path.abspath(local_path))
118     if os.path.basename(scratch).startswith("rally-input-") and os.path.isdir(scratch):
119         shutil.rmtree(scratch, ignore_errors=True)
120
121
122 def _backend_class(name: str):
123     """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
124     if name == "local":
125         from backend.storage.local import LocalStorage
126
127         return LocalStorage
128     if name == "s3":
129         from backend.storage.s3 import S3Storage
130
131         return S3Storage
132     if name == "gcs":
133         from backend.storage.gcs import GCSStorage
134
135         return GCSStorage
136     if name == "gdrive":
137         from backend.storage.gdrive import GDriveStorage
138
139         return GDriveStorage
140     if name == "gdrive-api":
141         from backend.storage.gdrive_api import GDriveApiStorage
142
143         return GDriveApiStorage
144     raise ValueError(
145         f"unknown storage backend: {name!r} (expected local|s3|gcs|gdrive|gdrive-api)"
146     )
147
148
149 def get_storage(
150     backend: Optional[str] = None,
151     *,
152     config: Optional[Dict[str, Any]] = None,
153     **overrides,
154 ) -> StorageBackend:
155     """Construct a backend. ``backend`` (or ``config['backend']``, default 'local')
156     selects
157     the implementation; its per-backend settings come from ``config[backend]`` merged
158     with
159     ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-
160     selectors."""
161     cfg = config or {}
162     name = backend or cfg.get("backend", "local")
163     # Per-backend settings live under the backend name; tolerate the hyphen?underscore
164     form so a
165     # hyphenated backend ("gdrive-api") can read a pydantic field that can't contain a
166     hyphen
167     # ("gdrive_api"). Hyphen-free names are unaffected (the first lookup wins).
168     settings: Dict[str, Any] = dict(
169         cfg.get(name) or cfg.get(name.replace("-", "_")) or {}
170     )
171     settings.update(overrides)
172     return _backend_class(name)(**settings)

```

```

169
170     def fetch(
171         uri: str,
172         dst: "str | os.PathLike[str] | None" = None,
173         *,
174         config: Optional[Dict[str, Any]] = None,
175         overwrite: bool = False,
176     ) -> str:
177         """Resolve ``uri`` to a local file path. For ``local`` this is an identity
passthrough
178         (no copy); for s3/gcs it downloads to ``dst``."""
179         ref = parse_uri(uri)
180         return get_storage(ref.backend, config=config).fetch_to_local(
181             ref, dst, overwrite=overwrite
182         )
183
184
185     def put(
186         local_path: "str | os.PathLike[str]",
187         uri: str,
188         *,
189         config: Optional[Dict[str, Any]] = None,
190         content_type: Optional[str] = None,
191     ) -> str:
192         """Upload/copy ``local_path`` to ``uri``; return ``uri``."""
193         ref = parse_uri(uri)
194         get_storage(ref.backend, config=config).put(
195             local_path, ref, content_type=content_type
196         )
197         return uri
198
199
200     def exists(uri: str, *, config: Optional[Dict[str, Any]] = None) -> bool:
201         """True if the object at ``uri`` exists (a bucket object or a local file)."""
202         ref = parse_uri(uri)
203         return bool(get_storage(ref.backend, config=config).exists(ref))
204
205
206     def signed_url(
207         uri: str,
208         *,
209         config: Optional[Dict[str, Any]] = None,
210         ttl: Optional[int] = None,
211         method: str = "GET",
212     ) -> str:
213         """A time-limited signed URL for ``uri`` ? the client fetches bytes DIRECTLY from
the store, off
214         the app's response path (so a large object isn't streamed back through the server;
#463). Backends
215         without object signing (e.g. local) raise ``NotImplementedError``.
216
217         ``ttl=None`` (the default) defers to the backend's configured ``signed_url_ttl``
from
218         ``config`` ? a hardcoded default here would silently shadow the operator's setting
(#469).
219         An explicit ``ttl`` overrides it for callers that need a specific expiry."""
220         ref = parse_uri(uri)
221         return get_storage(ref.backend, config=config).signed_url(ref, ttl=ttl,
method=method)
222
223
224     def list_uris(
225         prefix_uri: str, *, config: Optional[Dict[str, Any]] = None
226     ) -> Iterator[str]:
227         """Yield the URI of every object under ``prefix_uri``."""

```

```

228         ref = parse_uri(prefix_uri)
229         for r in get_storage(ref.backend, config=config).list(ref):
230             yield r.uri
231
232
233     def open_writer(
234         uri: str,
235         *,
236         config: Optional[Dict[str, Any]] = None,
237         content_type: Optional[str] = None,
238     ):
239         """A writable binary file-like streaming to ``uri`` (finalized on close). Backends
that
240         cannot stream raise ``NotImplementedError`` ? callers fall back to local staging
(seen
241         ``StorageBackend.open_writer``)."""
242         ref = parse_uri(uri)
243         return get_storage(ref.backend, config=config).open_writer(
244             ref, content_type=content_type
245         )
246

```

## 12. assistant (2026-07-09T04:23:54.933Z)

The ``\_\_init\_\_.py`` doesn't contain the GCSFuse helper ? it must be elsewhere. Let me search for the actual code and read the failing tests.

## 13. user (2026-07-09T04:23:58.320Z)

```

540
541     # ----- lazy-import hygiene
542
543
544     def test_importing_storage_does_not_pull_cloud_sdks():
545         # `import backend.storage` must not eagerly import boto3 / google-cloud-storage, so
the
546         # default install (and the CI lanes without those extras) stays clean.
547         code = (
548             "import sys, backend.storage; "
549             "assert 'boto3' not in sys.modules, 'boto3 imported eagerly'; "
550             "assert 'google.cloud.storage' not in sys.modules, 'gcs imported eagerly'; "
551             "print('clean')"
552         )
553         r = subprocess.run([sys.executable, "-c", code], capture_output=True, text=True)
554         assert r.returncode == 0, r.stderr
555         assert "clean" in r.stdout
556
557
558     def test_config_storage_defaults_are_non_breaking():
559         from backend.config import StorageConfig
560
561         sc = StorageConfig()
562         assert sc.backend == "local" and sc.output_root == ""
563         assert isinstance(storage.get_storage(config=sc.model_dump()), LocalStorage)
564

```

## 14. user (2026-07-09T04:23:59.114Z)

Found 74 files  
backend\pipeline\stitcher\compiler.py  
tests\test\_compiler.py  
backend\orchestration.py  
backend\main.py  
backend\config\models.py

docs\COMPUTE\_DECOUPLED\_SERVING\README.md  
docs\COMPUTE\_DECOUPLED\_SERVING\L4\_PRIMARY\_TOPOLOGY.md  
docs\COMPUTE\_DECOUPLED\_SERVING\runlogs\DEPLOY\_REPORT\_cloud-serving\_validation-latency\_2026-07-07.md  
backend\infrastructure\database.py  
deploy\cloudrun\README.md  
docs\CODE\_MAP.md  
docs\COMPUTE\_DECOUPLED\_SERVING\runlogs\DEPLOY\_REPORT\_cloud-serving\_gpu-resident-nvdec\_2026-07-07.md  
tests\test\_native\_wasb\_runner.py  
docs\PACKAGING\_PLAN.md  
deploy\digitalocean\gpu\_setup.sh  
docs\COMPUTE\_DECOUPLED\_SERVING\runlogs\DEPLOY\_REPORT\_cloud-serving\_alpha-alliv1\_2026-07-06.md  
backend\pipeline\segmenters\trajectory\_hybrid.py  
backend\pipeline\segmenters\gemini.py  
docs\CONTROL\_PLANE\_DURABLE\_REBUILD.md  
tests\test\_restart\_recovery.py  
docs\ALPHA\_LAUNCH\_REVIEW\_2026-07-04.md  
docs\COMPUTE\_DECOUPLED\_SERVING\runlogs\DEPLOY\_REPORT\_cloud-serving\_a1-e2e\_2026-07-04.md  
docs\COMPUTE\_DECOUPLED\_SERVING\runlogs\DEPLOY\_REPORT\_cloud-serving\_a1-cuj\_2026-07-04.md  
training\gen0\harness.py  
docs\COMPUTE\_DECOUPLED\_SERVING\runlogs\DEPLOY\_REPORT\_cloud-serving\_2026-07-04.md  
docs\archives\past\_projects\AUDIT\_REPORT\_khelsutra-guru\_2026-07-02-COMPLETED-2026-07-02.md  
tests\test\_app\_launcher.py  
CLAUDE.md  
docs\QUALITY\_ITERATIONS.md  
deploy\gcp\nightly\_regression\README.md  
tools\leak\_scan.py  
tests\test\_server\_entrypoint.py  
tests\test\_single\_instance.py  
tests\test\_personalization\_serving.py  
tests\test\_provider\_gemini.py  
tests\test\_p2b\_install.py  
tests\test\_per\_user\_gate.py  
tests\test\_golden\_real\_fixtures.py  
tests\test\_gemini.py  
tests\test\_db\_upgrade\_safety.py  
tests\test\_edge\_auth.py  
tests\test\_base.py  
scripts\uninstall.py  
backend\utils\single\_instance.py  
backend\providers\gemini.py  
backend\reporting\harvest.py  
backend\eval\per\_user\_regression.py  
backend\eval\regression.py  
backend\features\rally\_seq.py  
backend\eval\per\_user\_gate.py  
backend\eval\golden\_fixtures.py  
backend\eval\golden\_real\_fixtures.py  
backend\annotations\automated\_provider.py  
docs\archives\quality-iterations\README.md  
docs\archives\past\_projects\DECOUPLED\_COMPUTE\03-PLATFORM-OPTIONS.md  
docs\archives\past\_projects\DECOUPLED\_COMPUTE\04-BUILD-AND-TEST-PLAN.md  
docs\archives\field-pmf\PMF\_FIELD\_SIGNALS.md  
docs\archives\field-pmf\FIELD\_VISIT\_ANSWER\_SHEET.md  
docs\archives\decisions\DECISIONS.md  
docs\ROADMAP.md  
docs\PROMINENT\_COURT\_DETECTION.md  
docs\MULTI\_SIGNAL\_FUSION\_PLAN.md  
docs\GOLDEN\_REGRESSION\_FIXTURES.md  
docs\COMPUTE\_DECOUPLED\_SERVING\TESTING\_STRATEGY.md  
backend\results.py  
docs\VIDEO\_LOCALITY\_MODEL.md  
docs\PERSONALIZATION\_PLAN.md  
docs\archives\CODE\_AUDIT\_AND\_TEST\_HARDENING-COMPLETED-2026-06-14.md



docs\archives\DEFERRED\_HARDENING\_PLAN-COMPLETED-2026-06-14.md  
backend\reporting\spec.yaml  
training\README.md  
docs\archives\quality-iterations\2026-06-multivideo-lovo\README.md  
docs\archives\research\AUDIO\_RALLY\_DETECTION\_PLAN.md  
docs\archives\research\RALLY\_DETECTION\_RESEARCH.md

#### 15. assistant (2026-07-09T04:24:12.608Z)

The grep didn't match `test\_storage.py` or `backend/storage/` ? which is suspicious given the test is named `test\_gcs\_fuse\_helpers`. I'm currently on branch `codex/issue-523-sd-reel-plan-clear`, which may predate #535. Let me verify where the GCSFuse code actually lives and check the branch/master state.

#### 16. user (2026-07-09T04:24:14.682Z)

No matches found

#### 17. user (2026-07-09T04:24:19.082Z)

```
===FETCH===  
===535 on master?===  
8afbea9 perf(compile): use GCSFuse mount for source in worker to avoid full re-download (#535)  
(#541)  
===current branch vs origin/master===  
8      6
```

#### 18. assistant (2026-07-09T04:25:01.346Z)

Key finding: the current branch `codex/issue-523-sd-reel-plan-clear` **\*\*predates #535\*\*** ? it has no GCSFuse code (grep found no "fuse" in `test\_storage.py`). Meanwhile #535 is on `origin/master` (merged via #541: `8afbea9`), and the branch has diverged (origin/master is 8 ahead, branch is 6 ahead). The suite stats the user quoted (1933 passed/2 failed) match master, not this branch.

So the fix belongs on a branch off `origin/master`, not here. Let me inspect the actual code on master without disturbing this checkout yet.

#### 19. user (2026-07-09T04:25:03.968Z)

```
===where is fuse on master===  
origin/master:backend/annotations/automated_provider.py  
origin/master:backend/config/models.py  
origin/master:backend/eval/golden_fixtures.py  
origin/master:backend/eval/golden_real_fixtures.py  
origin/master:backend/eval/per_user_gate.py  
origin/master:backend/eval/per_user_regression.py  
origin/master:backend/eval/regression.py  
origin/master:backend/features/rally_seq.py  
origin/master:backend/infrastructure/database.py  
origin/master:backend/main.py  
origin/master:backend/orchestration.py  
origin/master:backend/pipeline/segmenters/gemini.py  
origin/master:backend/pipeline/segmenters/trajectory_hybrid.py  
origin/master:backend/pipeline/stitcher/compiler.py  
origin/master:backend/providers/gemini.py  
origin/master:backend/reporting/harvest.py  
origin/master:backend/reporting/spec.yaml  
origin/master:backend/results.py  
origin/master:backend/storage/__init__.py  
origin/master:backend/storage/capabilities.py  
origin/master:backend/utils/single_instance.py  
origin/master:tests/test_app_launcher.py  
origin/master:tests/test_base.py
```

```

origin/master:tests/test_compiler.py
origin/master:tests/test_db_upgrade_safety.py
origin/master:tests/test_edge_auth.py
origin/master:tests/test_gemini.py
origin/master:tests/test_golden_real_fixtures.py
origin/master:tests/test_native_wasb_runner.py
origin/master:tests/test_p2b_install.py
origin/master:tests/test_per_user_gate.py
origin/master:tests/test_personalization_serving.py
origin/master:tests/test_provider_gemini.py
origin/master:tests/test_restart_recovery.py
origin/master:tests/test_server_entrypoint.py
origin/master:tests/test_single_instance.py
origin/master:tests/test_storage.py
===commits 535/541===
3249451 feat(#534): live detector progress heartbeat + L4 worker resource telemetry (#546)
e29172a perf: gate CP acquire_local_input when validation+segment delegated to L4 (#539) (#545)
af54f2f fix(#538): cancel propagates to the dispatched L4 worker exec (#544)
14e4767 observability: emit structured signal on GPU encoder fallback to CPU (#529) (#543)
b68af0d docs: add verified bootstrap for reproducing tests locally (#542)
8afbea9 perf(compile): use GCSFuse mount for source in worker to avoid full re-download (#535)
(#541)
071d8b2 feat(compile): add opt-in SD reel rendition (#537)
f7a326d docs(runlogs): compile?L4 + NVENC live deploy-validation CUJ (2026-07-08) (#536)

```

## 20. assistant (2026-07-09T04:25:19.107Z)

On `origin/master` the GCSFuse code is in `backend/storage/\_\_init\_\_.py`, `backend/storage/capabilities.py`, and `tests/test\_storage.py`. Since this shared checkout is on an unrelated branch (possibly a sibling session's), I'll work in an isolated worktree off `origin/master`. First let me read the master code and the #535 diff, and understand the env setup.

## 21. user (2026-07-09T04:25:22.612Z)

```

===capabilities.py on master===
"""Honest, secret-free report of which storage media THIS box can use (STUDIO_MEDIA_LIFECYCLE.md
P3).

```

```

`GET /api/capabilities` surfaces this so the Studio medium picker (P5) renders only the
backends
this box can actually store into ? never a dead option. The contract mirrors the SPA mock
(`kheisutra/web/e2e/mock-engine.mjs` CAPS.storage): a ``default_medium`` + a list of
``{id, kind, label, usable, free_gb}``.

```

Rules (D14, D11):

- **local** ? always usable; **free\_gb** = free space on the output filesystem (best-effort).
- **gcs / s3** ? usable only when the cloud SDK imports AND the box has that bucket configured (a URI-shaped root or a non-empty per-backend settings dict). **free\_gb** is **None** ? a bucket has no readable capacity from here (cost/quota is warned elsewhere, not blocked).
- **gdrive** ? usable when a mounted Google Drive is resolvable (config / env / autodetect); **free\_gb** = free space on that mount.

**No secrets ever leave here** ? only presence/usable bools, labels, and free-space integers. Best-effort + never raises: an unresolvable probe degrades a field to **None**/**False**.

```

from __future__ import annotations

import importlib.util
import os
import shutil
from typing import Any, Dict, List, Optional

```

```

_BUCKET_LABELS = {
    "gcs": "Google Cloud Storage bucket",
    "s3": "Amazon S3 / compatible bucket",
}
_BUCKET_SDK = {"gcs": "google.cloud.storage", "s3": "boto3"}

def _as_dict(storage_config: Any) -> Dict[str, Any]:
    if storage_config is None:
        return {}
    if hasattr(storage_config, "model_dump"):
        return storage_config.model_dump() # type: ignore[no-any-return]
    return dict(storage_config)

def _free_gb(path: Optional[str]) -> Optional[int]:
    """Whole GB free on the filesystem holding ``path`` (best-effort; ``None`` if
    undeterminable).
    Walks up to the nearest existing ancestor (an output dir may not exist yet)."""
    if not path:
        return None
    probe = path
    for _ in range(64):
        try:
            return int(shutil.disk_usage(probe).free / (1024**3))
        except (OSError, ValueError):
            parent = os.path.dirname(probe.rstrip("/\\"))
            if not parent or parent == probe:
                return None
            probe = parent
    return None

def _sdk_importable(module: str) -> bool:
    """True if ``module`` is importable, WITHOUT importing it (cheap, never raises)."""
    try:
        return importlib.util.find_spec(module) is not None
    except (ImportError, ValueError, ModuleNotFoundError, AttributeError):
        return False

def _bucket_configured(sc: Dict[str, Any], name: str) -> bool:
    """Does the storage config name THIS bucket backend? (No secrets ? only scheme/selector
    hints.)"""
    if sc.get("backend") == name or sc.get("input_backend") == name:
        return True
    if sc.get(name): # a non-empty per-backend settings dict (endpoints/regions/paths)
        return True
    schemes = ("gs://", "gcs://") if name == "gcs" else (f"{name}://",)
    roots = (
        sc.get("input_root") or "",
        sc.get("output_root") or "",
        sc.get("workspace_root") or "",
    )
    return any(r.startswith(schemes) for r in roots)

def _bucket_backend(name: str, sc: Dict[str, Any]) -> Dict[str, Any]:
    usable = _sdk_importable(_BUCKET_SDK[name]) and _bucket_configured(sc, name)
    return {
        "id": name,
        "kind": name,
        "label": _BUCKET_LABELS[name],
        "usable": usable,
        "free_gb": None, # a bucket has no readable capacity from this box
    }

```

```

}

def _gdrive_mount_root(sc: Dict[str, Any]) -> Optional[str]:
    """Resolve a mounted Google Drive root (config ? env ? autodetect), or ``None``."""
    root = (sc.get("gdrive") or {}).get("mount_root") or os.environ.get(
        "GDRIVE_MOUNT_ROOT"
    )
    if root and os.path.isdir(root):
        return root
    try:
        from backend.storage.gdrive import _autodetect_mount_root

        auto = _autodetect_mount_root()
    except Exception: # noqa: BLE001 - detection is best-effort; never fail capabilities
        auto = None
    return auto if (auto and os.path.isdir(auto)) else None

def _gcs_mount_root(sc: Dict[str, Any]) -> Optional[str]:
    """Resolve a GCSFuse mount root for GCS inputs (config ? env ? common defaults), or
    ``None``.
    Used to enable range-reads of clip subsets via the mount instead of full-file download
    in compile workers (see #535). Mount is typically the bucket contents root (e.g. /gcs
    after `gcsfuse $BUCKET /gcs` so gs://$BUCKET/videos/x.mp4 appears at /gcs/videos/x.mp4)."""
    root = (sc.get("gcs") or {}).get("mount_root") or os.environ.get("GCS_MOUNT_ROOT")
    if root and os.path.isdir(root):
        return root
    # Common default in our L4 worker deploys (see deploy/cloudrun/README and runlogs).
    if os.path.isdir("/gcs"):
        return "/gcs"
    return None

def _gdrive_backend(sc: Dict[str, Any]) -> Dict[str, Any]:
    root = _gdrive_mount_root(sc)
    return {
        "id": "gdrive",
        "kind": "gdrive",
        "label": "Google Drive folder (mounted)",
        "usable": root is not None,
        "free_gb": _free_gb(root),
    }

def storage_capabilities(
    storage_config: Any, *, local_free_path: Optional[str] = None
) -> Dict[str, Any]:
    """Build the ``storage`` capabilities block (see module docstring). ``local_free_path`` is
    the
    directory whose free space represents "this computer" (the output/workspace root)."""
    sc = _as_dict(storage_config)
    backends: List[Dict[str, Any]] = [
        {
            "id": "local",
            "kind": "local",
            "label": "This computer",
            "usable": True,
            "free_gb": _free_gb(local_free_path or "."),
        },
        _bucket_backend("gcs", sc),
        _bucket_backend("s3", sc),
        _gdrive_backend(sc),
    ]
    return {"default_medium": "local", "backends": backends}

```

```
===ENV: venv?===
---python---
C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe
3.13.13 (tags/v3.13.13:01104ce, Apr 7 2026, 19:25:48) [MSC v.1944 64 bit (AMD64)]
```

## 22. assistant (2026-07-09T04:25:36.226Z)

Found `\_gcs\_mount\_root` in `capabilities.py` (returns the mount root only ? no path join). The actual path-join helper must be in `\_\_init\_\_.py`. Also important: **\*\*no in-repo venv\*\*** ? Python is global (3.13.13), so a worktree off master will share the same installed deps and can run the full suite. Let me read the #535 diff for the storage files and the two failing tests.

## 23. user (2026-07-09T04:25:37.125Z)

```
commit 8afbea95747904accb12a8d707643d6d0c41f894
Author: Avi <avi.dullu@gmail.com>
Date: Wed Jul 8 19:45:34 2026 +0530
```

perf(compile): use GCSFuse mount for source in worker to avoid full re-download (#535) (#541)

The L4 compile worker was doing a full SDK download of the source video (via `acquire_local_input + GCSStorage.fetch_to_local`) even for tiny subset reels. This took ~1m38s for a 5.8 GiB file when only ~80s of clips were needed.

- Add GCSFuse mount support for `gcs://` (modeled on `gdrive`):
  - `_gcs_mount_root + _resolve_gcs_fuse_path` helpers.
  - `acquire_local_input` now returns the fused path (e.g. `/gcs/...`) directly when usable ? no scratch dir, no full copy.
  - `input_is_local` and `cleanup` respect mounted paths.
- Forward `storage.gcs.mount_root=/gcs` (and support in `--set` overrides).
- Plumb rally range logging (`time_pairs`) and "using GCSFuse" messages into the compile path (for runlogs/debug as requested).
- Added unit tests.

Fixes #535. Implements the plan posted in the issue comment (LGTM received).

Primary path: workers now read only the requested clip ranges via the existing GCSFuse mount. Full-download fallback is preserved.

```
backend/config/models.py      | 2 ++
backend/orchestration.py      | 18 ++++++++
backend/storage/__init__.py   | 78 ++++++++++++++++++++++++++++++++++++++
backend/storage/capabilities.py | 14 ++++++
backend/worker/__main__.py    | 13 +++++-
tests/test_storage.py         | 53 ++++++++++++++++++++++++++++++++++
6 files changed, 172 insertions(+), 6 deletions(-)
=====DIFF: storage/__init__.py=====
commit 8afbea95747904accb12a8d707643d6d0c41f894
Author: Avi <avi.dullu@gmail.com>
Date: Wed Jul 8 19:45:34 2026 +0530
```

perf(compile): use GCSFuse mount for source in worker to avoid full re-download (#535) (#541)

The L4 compile worker was doing a full SDK download of the source video (via `acquire_local_input + GCSStorage.fetch_to_local`) even for tiny subset reels. This took ~1m38s for a 5.8 GiB file when only ~80s of clips were needed.

- Add GCSFuse mount support for `gcs://` (modeled on `gdrive`):
  - `_gcs_mount_root + _resolve_gcs_fuse_path` helpers.
  - `acquire_local_input` now returns the fused path (e.g. `/gcs/...`) directly when usable ? no scratch dir, no full copy.

- input\_is\_local and cleanup respect mounted paths.
- Forward storage.gcs.mount\_root=/gcs (and support in --set overrides).
- Plumb rally range logging (time\_pairs) and "using GCSFuse" messages into the compile path (for runlogs/debug as requested).
- Added unit tests.

Fixes #535. Implements the plan posted in the issue comment (LGTM received).

Primary path: workers now read only the requested clip ranges via the existing GCSFuse mount. Full-download fallback is preserved.

```
diff --git a/backend/storage/__init__.py b/backend/storage/__init__.py
index 8627fbf..592d871 100644
--- a/backend/storage/__init__.py
+++ b/backend/storage/__init__.py
@@ -13,6 +13,7 @@ passthrough. See ``docs/DECOUPLED_COMPUTE``.

from __future__ import annotations

+import logging
import os
import shutil
import tempfile
@@ -66,12 +67,59 @@ def resolve_input_ref(raw: str, storage_config: Any = None) -> StorageRef:
    return parse_uri(uri)

+def _gcs_mount_root(sc: Dict[str, Any]) -> Optional[str]:
+    """Resolve GCSFuse mount root (config/env/default /gcs) if usable. See #535.
+    When present, gs://bucket/key can be read as a local path under the mount (range seeks
+    via FUSE instead of full SDK download for compile subset stitches)."""
+    try:
+        from backend.storage.capabilities import _gcs_mount_root as _cap_gcs_root
+
+        return _cap_gcs_root(sc)
+    except Exception: # noqa: BLE001
+        pass
+    # fallback inline
+    root = (sc.get("gcs") or {}).get("mount_root") or os.environ.get("GCS_MOUNT_ROOT")
+    if root and os.path.isdir(root):
+        return root
+    if os.path.isdir("/gcs"):
+        return "/gcs"
+    return None
+
+def _resolve_gcs_fuse_path(ref: StorageRef, sc: Dict[str, Any]) -> Optional[str]:
+    """If GCS ref and a usable FUSE mount exists, return the local FS path under the mount.
+    Mapping: gs://$BUCKET/$KEY -> $mount_root/$KEY (GCSFuse mount typically exposes
+    bucket contents directly under mount root, as with our /gcs + WASB weights)."""
+    if ref.backend != "gcs":
+        return None
+    mount = _gcs_mount_root(sc)
+    if not mount:
+        return None
+    # key may have leading / from parse; join safely.
+    # We return the candidate path if the mount root is present; GCSFuse will serve bytes on
+    # open/seek (no need for the file to be pre-cached for exists() to succeed for large
+    videos).
+    key = ref.key.lstrip("/")
+    candidate = os.path.join(mount, key)
+    if os.path.isdir(mount):
+        return candidate
+    return None
+
```

```

+
def input_is_local(raw: str, storage_config: Any = None) -> bool:
    """True if ``raw`` resolves (under the input config) to a LOCAL filesystem path ? i.e. a
    plain on-disk file the caller can stat/validate directly. False for a bucket / Drive URI
    that must be fetched first. Raises ``ValueError`` for an unsupported scheme (same as
-   ``resolve_input_ref``)."""
-   return resolve_input_ref(raw, storage_config).backend == "local"
+   ``resolve_input_ref``).
+   For gcs://, also returns True if a GCSFuse mount makes it directly readable locally (#535).
+   """
+   ref = resolve_input_ref(raw, storage_config)
+   if ref.backend == "local":
+       return True
+   if ref.backend == "gcs":
+       sc = _storage_dict(storage_config)
+       if _resolve_gcs_fuse_path(ref, sc):
+           return True
+   return False

def acquire_local_input(
@@ -83,13 +131,29 @@ def acquire_local_input(
    byte-for-byte). A bucket / Drive source (``gs://``/``s3://``/``gdrive://``, or a bare name
under a
    non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir and the
local
    path returned. The caller owns successful remote scratch copies; use
-   ``cleanup_acquired_local_input`` when the consumer has finished reading the returned
path."""
+   ``cleanup_acquired_local_input`` when the consumer has finished reading the returned path.
+
+   For gcs://, if a GCSFuse mount_root is configured and the object is readable under the
mount,
+   the fused local path is returned directly (no download/scratch). This enables range/seek
reads of only the needed rally clips in compile (#535)."""
    sc = _storage_dict(storage_config)
    ref = resolve_input_ref(raw, sc)
    if ref.backend == "local":
        return (
            ref.key
        ) # identity ? no temp, no copy, byte-identical to passing the path directly
+
+   # GCS FUSE support (#535): if a mount root is configured and usable for this ref,
+   # return the fused local path directly. This lets the stitcher (ffmpeg) perform
+   # range/seek reads of only the needed clip windows instead of a full download.
+   # Falls back to full SDK fetch if no mount or path not resolvable.
+   if ref.backend == "gcs":
+       fused = _resolve_gcs_fuse_path(ref, sc)
+       if fused:
+           logger = logging.getLogger(__name__)
+           logger.info("[storage] using GCSFuse mounted path for gcs source (no full
download): %s", fused)
+           return fused
+
    scratch = tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
    try:
        return fetch(ref.uri, os.path.join(scratch, ref.name or "in.mp4"), config=sc)
@@ -106,6 +170,7 @@ def cleanup_acquired_local_input(
    Local inputs are identity paths and must never be removed. Remote acquisitions are
downloaded
    into a fresh ``rally-input-*`` directory; deleting that directory is safe once the caller
has
    finished hashing/validating/stitching/segmenting the returned file.
+   Fused GCS paths (via GCSFuse mount) are never cleaned (they are not our scratch).
    """

```

```

try:
    if input_is_local(raw, storage_config):
@@ -114,6 +179,13 @@ def cleanup_acquired_local_input(
    return
    if not local_path:
        return
+   # Do not clean paths that live under a GCS FUSE mount (they are not rally-input- scratch).
+   sc = _storage_dict(storage_config)
+   mount = _gcs_mount_root(sc)
+   if mount:
+       abs_path = os.path.abspath(local_path)
+       if abs_path.startswith(os.path.abspath(mount) + os.sep) or abs_path ==
os.path.abspath(mount):
+       return
        scratch = os.path.dirname(os.path.abspath(local_path))
        if os.path.basename(scratch).startswith("rally-input-") and os.path.isdir(scratch):
            shutil.rmtree(scratch, ignore_errors=True)

```

#### 24. assistant (2026-07-09T04:26:20.821Z)

Now I can see the exact bug. In `\_resolve\_gcs\_fuse\_path`:

```

```python
key = ref.key.lstrip("/")          # key = "videos/clip.mp4" (POSIX, from the gs:// URI)
candidate = os.path.join(mount, key) # Windows: "C:\...\mount" + "\" + "videos/clip.mp4"
...

`os.path.join` inserts an OS-native separator between `mount` and `key`, but the **forward slash
embedded inside `key` is preserved verbatim ? mixed separators on Windows
(`...\videos/clip.mp4`), while the test's `str(mount / "videos/clip.mp4")` is fully OS-native
(`...\videos\clip.mp4`). On Linux both are `/`, so CI is green.

```

Let me read the exact test code to confirm both failures and validate my fix against them.

#### 25. user (2026-07-09T04:26:23.323Z)

```

commit 8afbea95747904accb12a8d707643d6d0c41f894
Author: Avi <avi.dullu@gmail.com>
Date:   Wed Jul 8 19:45:34 2026 +0530

```

perf(compile): use GCSFuse mount for source in worker to avoid full re-download (#535)  
(#541)

The L4 compile worker was doing a full SDK download of the source video  
(via acquire\_local\_input + GCSStorage.fetch\_to\_local) even for tiny  
subset reels. This took ~1m38s for a 5.8 GiB file when only ~80s of  
clips were needed.

- Add GCSFuse mount support for gcs:// (modeled on gdrive):
  - \_gcs\_mount\_root + \_resolve\_gcs\_fuse\_path helpers.
  - acquire\_local\_input now returns the fused path (e.g. /gcs/...) directly  
when usable ? no scratch dir, no full copy.
  - input\_is\_local and cleanup respect mounted paths.
- Forward storage.gcs.mount\_root=/gcs (and support in --set overrides).
- Plumb rally range logging (time\_pairs) and "using GCSFuse" messages  
into the compile path (for runlogs/debug as requested).
- Added unit tests.

Fixes #535. Implements the plan posted in the issue comment (LGTM received).

Primary path: workers now read only the requested clip ranges via the  
existing GCSFuse mount. Full-download fallback is preserved.

```

diff --git a/tests/test_storage.py b/tests/test_storage.py
index 12e04b5..d68b50b 100644
--- a/tests/test_storage.py
+++ b/tests/test_storage.py

```



```

@@ -561,3 +561,56 @@ def test_config_storage_defaults_are_non_breaking():
    sc = StorageConfig()
    assert sc.backend == "local" and sc.output_root == ""
    assert isinstance(storage.get_storage(config=sc.model_dump()), LocalStorage)
+
+
+# ----- GCS FUSE mount support for range reads
+#535
+
+
+def test_gcs_fuse_helpers(tmp_path):
+    mount = tmp_path / "gcs"
+    mount.mkdir()
+    (mount / "videos").mkdir()
+
+    sc = {"gcs": {"mount_root": str(mount)}}
+    assert storage._gcs_mount_root(sc) == str(mount)
+
+    ref = storage.parse_uri("gs://bucket/videos/clip.mp4")
+    fused = storage._resolve_gcs_fuse_path(ref, sc)
+    assert fused == str(mount / "videos/clip.mp4")
+
+    # no mount -> None
+    assert storage._resolve_gcs_fuse_path(ref, {}) is None
+
+
+def test_acquire_local_input_uses_gcs_fuse_when_available(tmp_path, monkeypatch):
+    mount = tmp_path / "gcs"
+    mount.mkdir()
+    (mount / "videos").mkdir()
+    clip = mount / "videos/clip.mp4"
+    clip.write_bytes(b"fake video bytes for fuse test")
+
+    sc = {"gcs": {"mount_root": str(mount)}, "backend": "gcs"}
+
+    # simulate gcs ref
+    path = storage.acquire_local_input("gs://bucket/videos/clip.mp4", sc)
+    assert path == str(clip)
+    assert path.startswith(str(mount))
+
+    # cleanup should be no-op for fused path
+    storage.cleanup_acquired_local_input("gs://bucket/videos/clip.mp4", path, sc)
+    assert clip.exists() # not deleted
+
+
+def test_acquire_falls_back_to_fetch_without_mount(monkeypatch):
+    calls = []
+    def fake_fetch(uri, dst, *, config=None):
+        calls.append(uri)
+        os.makedirs(os.path.dirname(dst), exist_ok=True)
+        open(dst, "wb").write(b"downloaded")
+        return dst
+    monkeypatch.setattr(storage, "fetch", fake_fetch)
+
+    sc = {"backend": "gcs"} # no mount_root
+    p = storage.acquire_local_input("gs://b/vid.mp4", sc)
+    assert "downloaded" in open(p, "rb").read().decode()
+    assert calls == ["gs://b/vid.mp4"]

```

## 26. assistant (2026-07-09T04:28:01.424Z)

Both failing tests confirmed. **Both stem from the same single line** ? ``os.path.join(mount, key)`` in ``_resolve_gcs_fuse_path``, where ``key = "videos/clip.mp4"`` keeps its embedded POSIX slash, while the tests compare against pathlib's OS-native ``str(mount / "videos/clip.mp4")``.

The correct contract is **\*\*Option A (OS-native)\*\***: a GCSFuse mount is a real local filesystem path, and the code already uses ``os.path.join`` (not a hardcoded ``/``) ? the only defect is that the POSIX object key isn't converted to native separators. Fixing it is byte-identical on Linux (`os.sep` is ``/``) and makes both tests pass on Windows as-written, keeping cross-platform coverage. Option B (POSIX + linux-only test) would leave the helper Windows-broken for no benefit.

Now let me set up an **\*\*isolated worktree off `origin/master`\*\*** (this shared checkout is on unrelated branch ``codex/issue-523`` with a possible sibling session's untracked file ? I won't disturb it). No in-repo venv means the worktree shares global Python + deps, so the full suite runs there.

## 27. user (2026-07-09T04:28:04.867Z)

```
===worktrees===
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer
a460d74 [codex/issue-523-sd-reel-plan-clear]
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/cfg-wt          20b2d5a [claude/wasb-
config-validation]
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/compile-wt      b09a1d4
[claude/compile-reliability]
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/pr458-pre-wt    355dad6 (detached
HEAD)
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/6e9d0958-a2ec-4a8e-bb2d-6cce9833c527/scratchpad/pr458-wt        5de1ef2 (detached
HEAD)
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/744270aa-6a38-4011-b018-cdd9aae94849/scratchpad/perfbench/wt-dev 0aa6d4b [feat/perf-
release-gate]
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/744270aa-6a38-4011-b018-cdd9aae94849/scratchpad/perfbench/wt/prod 304af7b (detached
HEAD)
C:/Users/avidu/AppData/Local/Temp/claude/c--Users-avidu-Projects-badminton-highlight-
indexer/744270aa-6a38-4011-b018-cdd9aae94849/scratchpad/perfbench/wt/rc  3249451 (detached
HEAD)
C:/Users/avidu/Projects/khelsutra-guru/bhi-wt-cheap-watermark
65ee4c6 [codex/issue-523-sd-reel-review]
===origin/master sha===
3249451
===is current branch pushed?===
a460d74
  origin/codex/issue-523-sd-reel-plan-clear
===untracked file mtime===
-rw-r--r-- 1 avidu 197609 2282 Jul  8 10:53 SESSION_HANDOFF (1).md
```

## 28. assistant (2026-07-09T04:28:29.828Z)

The current branch is safely pushed to origin, and there are already **\*\*many\*\*** concurrent-session worktrees ? confirming heavy multi-session activity. A worktree is definitely correct. ``origin/master`` = ``3249451``. Let me create an isolated worktree off ``origin/master`` in my session scratchpad.

## 29. user (2026-07-09T04:28:35.051Z)

```
Preparing worktree (new branch 'claude/gcsfuse-native-path')
branch 'claude/gcsfuse-native-path' set up to track 'origin/master'.
HEAD is now at 3249451 feat(#534): live detector progress heartbeat + L4 worker resource
telemetry (#546)
===VERIFY===
claude/gcsfuse-native-path
3249451 feat(#534): live detector progress heartbeat + L4 worker resource telemetry (#546)
```

```
===confirm fuse code present===
backend/storage/__init__.py:3
tests/test_storage.py:2
```

### 30. assistant (2026-07-09T04:28:39.486Z)

Worktree is on `claude/gcsfuse-native-path` at `origin/master` (3249451), with the fuse code present. Let me first **\*\*reproduce\*\*** the two failures in this worktree to confirm it faithfully mirrors the user's baseline before I touch anything.

### 31. user (2026-07-09T04:28:44.638Z)

```
_____ test_acquire_local_input_uses_gcs_fuse_when_available _____

tmp_path = WindowsPath('C:/Users/avidu/AppData/Local/Temp/pytest-of-
avidu/pytest-1687/test_acquire_local_input_uses_0')
monkeypatch = <_pytest.monkeypatch.MonkeyPatch object at 0x000001CEB23D6030>

def test_acquire_local_input_uses_gcs_fuse_when_available(tmp_path, monkeypatch):
    mount = tmp_path / "gcs"
    mount.mkdir()
    (mount / "videos").mkdir()
    clip = mount / "videos/clip.mp4"
    clip.write_bytes(b"fake video bytes for fuse test")

    sc = {"gcs": {"mount_root": str(mount)}, "backend": "gcs"}

    # simulate gcs ref
    path = storage.acquire_local_input("gs://bucket/videos/clip.mp4", sc)
> assert path == str(clip)
E   AssertionError: assert 'C:\\Users\\a...deos/clip.mp4' == 'C:\\Users\\a...eos\\clip.mp4'
E
E       Skipping 94 identical leading characters in diff, use -v to show
E       - gcs\\videos\\clip.mp4
E       ?           ^
E       + gcs\\videos/clip.mp4
E       ?           ^

tests\\test_storage.py:596: AssertionError
===== short test summary info =====
FAILED tests/test_storage.py::test_gcs_fuse_helpers - AssertionError: assert ...
FAILED tests/test_storage.py::test_acquire_local_input_uses_gcs_fuse_when_available
2 failed, 39 passed in 1.77s
```

### 32. assistant (2026-07-09T04:28:59.475Z)

Reproduced exactly: `2 failed, 39 passed`, and the diff confirms the diagnosis ?  
`gcs\\videos/clip.mp4` (code, mixed separators) vs `gcs\\videos\\clip.mp4` (expected, native). The worktree faithfully mirrors the baseline.

Now let me read the exact current code in the worktree before editing (there may be post-#535 changes on master).

### 33. user (2026-07-09T04:29:00.525Z)

```
70     def _gcs_mount_root(sc: Dict[str, Any]) -> Optional[str]:
71         """Resolve GCSFuse mount root (config/env/default /gcs) if usable. See #535.
72         When present, gs://bucket/key can be read as a local path under the mount (range
seeks
73         via FUSE instead of full SDK download for compile subset stitches)."""
74         try:
75             from backend.storage.capabilities import _gcs_mount_root as _cap_gcs_root
76
77             return _cap_gcs_root(sc)
```

```

78         except Exception: # noqa: BLE001
79             pass
80         # fallback inline
81         root = (sc.get("gcs") or {}).get("mount_root") or os.environ.get("GCS_MOUNT_ROOT")
82         if root and os.path.isdir(root):
83             return root
84         if os.path.isdir("/gcs"):
85             return "/gcs"
86         return None
87
88
89     def _resolve_gcs_fuse_path(ref: StorageRef, sc: Dict[str, Any]) -> Optional[str]:
90         """If GCS ref and a usable FUSE mount exists, return the local FS path under the
mount.
91         Mapping: gs://$BUCKET/$KEY -> $mount_root/$KEY (GCSFuse mount typically exposes
92         bucket contents directly under mount root, as with our /gcs + WASB weights)."""
93         if ref.backend != "gcs":
94             return None
95         mount = _gcs_mount_root(sc)
96         if not mount:
97             return None
98         # key may have leading / from parse; join safely.
99         # We return the candidate path if the mount root is present; GCSFuse will serve
bytes on
100         # open/seek (no need for the file to be pre-cached for exists() to succeed for large
videos).
101         key = ref.key.lstrip("/")
102         candidate = os.path.join(mount, key)
103         if os.path.isdir(mount):
104             return candidate
105         return None
106
107
108     def input_is_local(raw: str, storage_config: Any = None) -> bool:
109         """True if ``raw`` resolves (under the input config) to a LOCAL filesystem path ?
i.e. a
110         plain on-disk file the caller can stat/validate directly. False for a bucket / Drive
URI
111         that must be fetched first. Raises ``ValueError`` for an unsupported scheme (same as
112         ``resolve_input_ref``).
113         For gcs://, also returns True if a GCSFuse mount makes it directly readable locally
(#535).
114         """
115         ref = resolve_input_ref(raw, storage_config)
116         if ref.backend == "local":
117             return True
118         if ref.backend == "gcs":
119             sc = _storage_dict(storage_config)
120             if _resolve_gcs_fuse_path(ref, sc):
121                 return True
122         return False
123
124

```

#### 34. assistant (2026-07-09T04:29:30.005Z)

Code matches #535 exactly (unchanged since merge). Now applying the **Option A (OS-native)** fix  
? convert the POSIX object key into path segments so `os.path.join` builds a native path:

#### 35. user (2026-07-09T04:29:36.808Z)

The file C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/d596f7ef-c2b1-4b96-b022-918ac3e40023/scratchpad/gcsfuse-wt/backend/storage/\_\_init\_\_.py has been updated successfully. (file state is current in your context ? no need to Read it back)

### 36. assistant (2026-07-09T04:29:39.411Z)

Fix applied. Let me verify the two target tests (plus the full `test\_storage.py` module) now pass:

### 37. user (2026-07-09T04:29:47.480Z)

```
cachedir: .pytest_cache
rootdir: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\d596f7ef-c2b1-4b96-b022-918ac3e40023\scratchpad\gcsfuse-wt
configfile: pytest.ini
plugins: anyio-4.13.0, asyncio-0.26.0, cov-6.3.0
asyncio: mode=Mode.STRICT, asyncio_default_fixture_loop_scope=None,
asyncio_default_test_loop_scope=function
collecting ... collected 3 items

tests/test_storage.py::test_gcs_fuse_helpers PASSED [ 33%]
tests/test_storage.py::test_acquire_local_input_uses_gcs_fuse_when_available PASSED [ 66%]
tests/test_storage.py::test_acquire_falls_back_to_fetch_without_mount PASSED [100%]

===== 3 passed in 0.24s =====
===FULL test_storage.py module===
..... [100%]
41 passed in 1.26s
```

### 38. assistant (2026-07-09T04:30:03.281Z)

Both target tests now pass, and the full `test\_storage.py` module is `41 passed` (was `2 failed, 39 passed`). Now the user's required gate: the full suite. Let me quickly check what `run\_all\_tests.py` does (parallelism, arg passthrough) before launching it.

### 39. user (2026-07-09T04:30:05.993Z)

```
#!/usr/bin/env python
"""Run the FULL test suite for the badminton-highlight-indexer.
```

A single, discoverable entry point so you (or a reviewer / CI) can verify the whole suite before opening a PR:

```
python run_all_tests.py          # full suite, quiet
python run_all_tests.py -v       # verbose (passthrough to pytest)
python run_all_tests.py -k config # filter by keyword (passthrough)
python run_all_tests.py --cov=backend --cov-report=term-missing # coverage
```

Anything after the script name is passed straight through to pytest.

#### Notes

```
-----
- The suite is fully mocked (no GPU, no network, no WSL) and runs offline.
- The observability **reporting** tests skip cleanly if the optional ``obsreport``
  package isn't installed (they use ``pytest.importorskip``). To exercise them too:
  pip install -e ../sports-obsreport --no-build-isolation
- Exit code is pytest's own, so this is CI / pre-PR friendly (0 = all green).
"""
```

```
from __future__ import annotations
```

```
import importlib.util
import subprocess
import sys
```

```
def _installed(module: str) -> bool:
    try:
```

```

    return importlib.util.find_spec(module) is not None
except (ImportError, ValueError):
    return False

def main() -> int:
    args = list(sys.argv[1:])
    # Default to quiet unless the caller chose a verbosity.
    if not any(a in ("-q", "-v", "-vv", "-vvv", "--quiet", "--verbose") for a in args):
        args = ["-q", *args]
    cmd = [sys.executable, "-m", "pytest", *args]

    obs = (
        "available"
        if _installed("obsreport")
        else "NOT installed ? reporting tests will SKIP"
    )
    bar = "=" * 72
    print(bar)
    print(" Full test suite: " + " ".join(cmd))
    print(f" obsreport (optional reporting dep): {obs}")
    print(bar, flush=True)

    rc = subprocess.run(cmd).returncode

    print(bar)
    print(
        " RESULT: "
        + ("PASS - all tests green" if rc == 0 else f"FAIL - pytest exit {rc}")
    )
===xdist installed?===
Traceback (most recent call last):

```

#### 40. assistant (2026-07-09T04:30:20.679Z)

`run\_all\_tests.py` passes args straight to pytest (no xdist, so serial). This is the full ~1935-test suite and may take several minutes, so I'll run it in the background with output logged, and be notified on completion.

#### 41. user (2026-07-09T04:30:22.806Z)

Command running in background with ID: bial355gf. Output is being written to: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\d596f7ef-c2b1-4b96-b022-918ac3e40023\tasks\bial355gf.output. You will be notified when it completes. To check interim output, use Read on that file path.

#### 42. assistant (2026-07-09T04:30:37.919Z)

The suite runs in the background. Meanwhile, let me validate my change against the repo's CI static checks (ruff + mypy) ? these are independent read-only processes. My edit adds no imports and keeps the same return type, so it should be clean, but let me confirm.

#### 43. user (2026-07-09T04:30:47.906Z)

```

===ruff version + check changed file===
ruff 0.15.16
All checks passed!
===mypy changed module===
mypy 2.1.0 (compiled: yes)
Success: no issues found in 1 source file

```

#### 44. assistant (2026-07-09T04:30:53.938Z)

ruff 0.15.16 and mypy 2.1.0 (both matching CI versions) pass clean on the changed file. Let me peek at the suite's interim output to confirm it's collecting/running properly (not erroring on

collection).

#### 45. user (2026-07-09T04:31:04.821Z)

```
===interim fullsuite.log tail===
```

```
=====
Full test suite: C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe -m pytest
-q --cov=backend --cov=training --cov-fail-under=70
obsreport (optional reporting dep): available
=====
.....---
dots-so-far (rough progress):
8
```

#### 46. assistant (2026-07-09T04:31:31.883Z)

The suite is running cleanly (`obsreport` is available here, so the optional reporting tests actually execute ? meaning my worktree runs a **superset** of the user's baseline, not a subset). Static checks are already green:

- **ruff 0.15.16** (CI version) ? All checks passed
- **mypy 2.1.0** (CI version) ? no issues
- **Target tests** ? both now pass; `test\_storage.py` = 41 passed (was 2 failed / 39 passed)

I'll wait for the full-suite background run to finish (I'm notified on completion ? no polling) and then report the final pass/fail count and coverage.

```
**The fix** ([backend/storage/__init__.py](backend/storage/__init__.py), in
`_resolve_gcs_fuse_path`):
```

```
```python
```

**before ? embedded "/" in the POSIX object key survives on Windows ? "?gcs\videos/clip.mp4"**

```
key = ref.key.lstrip("/")
candidate = os.path.join(mount, key)
```

**after ? split the POSIX key into segments so os.path.join builds a native path**

```
segments = [seg for seg in ref.key.split("/") if seg]
candidate = os.path.join(mount, *segments)
```
```

**\*\*Contract decision:\*\*** Option A (OS-native). A GCSFuse mount is a real local filesystem, so the fused path must use the OS separator. On Linux `os.sep == "/"`, so the output is byte-identical to before (production is unaffected); on Windows it now yields `?gcs\videos\clip.mp4`, matching the tests' `str(mount / "videos/clip.mp4")` oracle without making them Linux-only ? so cross-platform coverage is preserved.

#### 47. user (2026-07-09T04:31:32.657Z)

<system-reminder>Warning: the file exists but is shorter than the provided offset (1). The file has 1 lines.</system-reminder>

#### 48. assistant (2026-07-09T04:31:47.232Z)

The full-suite run is in the background; its completion will notify me. I'll report the final numbers (pass/fail count + coverage) as soon as it finishes ? nothing more to do until then.

#### 49. user (2026-07-09T04:38:19.768Z)

```
<task-notification>
<task-id>bial355gf</task-id>
<tool-use-id>toolu_01LvQNDDDeUnacgCiDrZNstKg</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-
highlight-indexer\d596f7ef-c2b1-4b96-b022-918ac3e40023\tasks\bial355gf.output</output-file>
```

```
<status>completed</status>
<summary>Background command "Run full test suite with coverage (background)" completed (exit
code 0)</summary>
</task-notification>
```

#### 50. assistant (2026-07-09T04:38:26.719Z)

Suite completed with **exit code 0** (pytest green AND coverage floor met). Let me read the final summary and coverage totals from the log.

#### 51. user (2026-07-09T04:38:30.661Z)

```
===final result lines===
C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-packages\yaml\error.py:6:
ResourceWarning: unclosed database in <sqlite3.Connection object at 0x000001F72ED932E0>
C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-packages\yaml\error.py:6:
ResourceWarning: unclosed database in <sqlite3.Connection object at 0x000001F730398040>
backend\results.py                27      0   100%
TOTAL                          13541   2128    84%
Required test coverage of 70% reached. Total coverage: 84.28%
1935 passed, 8 skipped, 57 warnings in 472.51s (0:07:52)
  RESULT: PASS - all tests green
===confirm target tests not in any failure===
```

#### 52. assistant (2026-07-09T04:38:58.008Z)

Let me capture the exact diff to present.

#### 53. user (2026-07-09T04:39:01.138Z)

```
===diff vs origin/master===
 backend/storage/__init__.py | 10 ++++++---
 1 file changed, 7 insertions(+), 3 deletions(-)

diff --git a/backend/storage/__init__.py b/backend/storage/__init__.py
index 592d871..f9a0a92 100644
--- a/backend/storage/__init__.py
+++ b/backend/storage/__init__.py
@@ -95,11 +95,15 @@ def _resolve_gcs_fuse_path(ref: StorageRef, sc: Dict[str, Any]) ->
Optional[str]
     mount = _gcs_mount_root(sc)
     if not mount:
         return None
-    # key may have leading / from parse; join safely.
+    # ref.key is a POSIX ("/"-separated) object key; split it into segments so os.path.join
+    builds
+    # an OS-native local path (backslash-separated on Windows). A GCSFuse mount is a real local
+    # filesystem, so the fused path must use the native separator ? a bare os.path.join(mount,
+    key)
+    # would leave the embedded "/" verbatim and only match a native path on POSIX hosts (#535).
+    # A leading "/" from parse and any empty segments are dropped.
+    # We return the candidate path if the mount root is present; GCSFuse will serve bytes on
+    # open/seek (no need for the file to be pre-cached for exists()) to succeed for large
+    videos).
-    key = ref.key.lstrip("/")
-    candidate = os.path.join(mount, key)
+    segments = [seg for seg in ref.key.split("/") if seg]
+    candidate = os.path.join(mount, *segments)
+    if os.path.isdir(mount):
```



```
    return candidate
return None
```

#### 54. assistant (2026-07-09T04:39:35.144Z)

Done. The fix is verified end-to-end on Windows.

#### Root cause

In ``_resolve_gcs_fuse_path`` (`[backend/storage/___init__.py](backend/storage/___init__.py)`), the GCS object key from the ``gs://`` URI is POSIX (``videos/clip.mp4``). The code did ``os.path.join(mount, key)``, which inserts a native separator between ``mount`` and ``key`` but leaves the ``**embedded**`` ``/`` untouched ? on Windows the fused path was mixed (``?gcs\videos\clip.mp4``), while the tests' oracle ``str(mount / "videos/clip.mp4")`` is fully native (``?gcs\videos\clip.mp4``). On Linux ``os.sep == "/"``, so both sides match and CI stays green ? a latent portability bug from #535.

#### Fix ? Option A (OS-native), the correct contract

A GCSFuse mount is a real local filesystem, so the fused path must use the OS separator. Split the POSIX key into segments and let ``os.path.join`` build the native path:

```
```python
segments = [seg for seg in ref.key.split("/") if seg]
candidate = os.path.join(mount, *segments)
```
```

This is ``**byte-identical on Linux`` (``os.sep == "/"``, so production/L4 workers are unaffected) and yields a native path on Windows, so both tests pass ``**as written`` ? no need to make them Linux-only, preserving cross-platform coverage. (1 file, +7/?3.) I rejected Option B: keeping POSIX + a linux-only test would leave the helper Windows-broken for no benefit.

#### Verification (Windows, this worktree)

```
| Check | Result |
|---|---|
| `run_all_tests.py --cov=backend --cov=training --cov-fail-under=70` | **1935 passed, 8
skipped, 0 failed** (7:52), exit 0 |
| vs. baseline | was **1933 passed / 2 failed** ? the 2 target tests now pass |
| Coverage | **84.28%** (? 70 floor; **no regression** vs. baseline) |
| `test_gcs_fuse_helpers`, `test_acquire_local_input_uses_gcs_fuse_when_available` | **PASS** |
| ruff 0.15.16 (`ruff check`) | clean |
| mypy 2.1.0 | clean |
```

``obsreport`` was installed here, so the optional reporting tests actually ran ? the worktree exercised a ``**superset**` of your baseline, not a subset.

#### Where it lives / next step

The change is uncommitted on branch ``**`claude/gcsfuse-native-path`**` (cut from ``origin/master`` @ ``3249451``) in an isolated worktree ? I deliberately did ``**not**` touch your shared checkout, which is on ``codex/issue-523-sd-reel-plan-clear`` (a sibling session, plus a stray untracked ``SESSION_HANDOFF (1).md`` I left alone).

Since this is unrelated to PR #547, the natural landing is a small standalone PR. Want me to ``**commit it and open a PR against master``? (I haven't committed yet ? say the word and I'll do the branch-safety re-check, commit the single file, and open it.)

#### 55. user (2026-07-09T04:42:20.309Z)

pls open a PR and send for review

#### 56. assistant (2026-07-09T04:42:49.420Z)

On it. First the branching-safety re-checks (per my concurrent-session protocol) plus confirming ``gh`` is available and authenticated.

## 57. user (2026-07-09T04:42:58.499Z)

```
===refetch origin===
===branch (must be mine)===
claude/gcsfuse-native-path
===HEAD (must be mine; pre-commit == master base)===
3249451 feat(#534): live detector progress heartbeat + L4 worker resource telemetry (#546)
===working tree (only the one file)===
  M backend/storage/__init__.py
===origin/master now===
3249451
===gh===
gh version 2.92.0 (2026-04-28)
github.com
  ? Logged in to github.com account avidullu (keyring)
  - Active account: true
```

## 58. assistant (2026-07-09T04:43:24.068Z)

All green: branch `claude/gcsfuse-native-path` (mine), HEAD at base `3249451`, only the one file modified, origin/master unmoved, `gh` authenticated as `avidullu`. Let me write the commit message and PR body to files (avoids shell-quoting issues), then commit the explicit path.

## 59. assistant (2026-07-09T04:43:30.136Z)

fix(storage): build GCSFuse fused path with OS-native separators

\_resolve\_gcs\_fuse\_path joined the mount root with the raw POSIX object key via os.path.join(mount, key), leaving the key's embedded "/" verbatim. On Windows that produced a mixed-separator path (...\\gcs\\videos\\clip.mp4) while callers/tests expect a native path (...\\gcs\\videos\\clip.mp4). On Linux os.sep == "/" so both matched and CI stayed green -- a latent portability bug from #535 (GCSFuse mount for compile source).

A GCSFuse mount is a real local filesystem, so the fused path must use the OS-native separator. Split the POSIX key into segments and let os.path.join build the native path: os.path.join(mount, \*segments). Byte-identical on Linux (production / L4 workers unaffected); yields a native path on Windows, so the existing tests pass as written -- no need to mark them Linux-only, and cross-platform coverage is preserved.

Fixes the two local-Windows failures:

- tests/test\_storage.py::test\_gcs\_fuse\_helpers
- tests/test\_storage.py::test\_acquire\_local\_input\_uses\_gcs\_fuse\_when\_available

Verified on Windows: full suite 1935 passed / 8 skipped / 0 failed, coverage 84.28% (floor 70), ruff 0.15.16 + mypy 2.1.0 clean.

Co-Authored-By: Claude Opus 4.8 <noreply@anthropic.com>

## 60. user (2026-07-09T04:43:30.190Z)

File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/d596f7ef-c2b1-4b96-b022-918ac3e40023/scratchpad/commit\_msg.txt (file state is current in your context ? no need to Read it back)

## 61. assistant (2026-07-09T04:43:42.029Z)

### Summary

Fixes a latent **Windows portability bug** from #535 (`perf(compile): use GCSFuse mount for source in worker`). Two tests fail on local Windows while passing on CI Linux, so `master` CI is green and this went unnoticed:

```
- `tests/test_storage.py::test_gcs_fuse_helpers`  
- `tests/test_storage.py::test_acquire_local_input_uses_gcs_fuse_when_available`
```

## Root cause

`\_resolve\_gcs\_fuse\_path` (`backend/storage/\_\_init\_\_.py`) mapped `gs://\$BUCKET/\$KEY` onto the mount with:

```
```python  
key = ref.key.lstrip("/")          # POSIX object key, e.g. "videos/clip.mp4"  
candidate = os.path.join(mount, key)  
```
```

`os.path.join` inserts an OS-native separator between `mount` and `key`, but the **\*\*embedded\*\*** `/` inside the key is left verbatim. On Windows that yields a mixed-separator path (`...\gcs\videos\clip.mp4`), while callers and the tests expect a native path (`...\gcs\videos\clip.mp4`, i.e. `str(mount / "videos/clip.mp4")`). On Linux `os.sep == "/"`, so both forms are identical and CI stays green.

## Fix ? OS-native path (the correct contract)

A GCSFuse mount is a real local filesystem, so the fused path must use the OS-native separator. Split the POSIX key into segments and let `os.path.join` build the native path:

```
```python  
segments = [seg for seg in ref.key.split("/") if seg]  
candidate = os.path.join(mount, *segments)  
```
```

- **\*\*Byte-identical on Linux\*\*** (`os.sep == "/"`) ? production and the L4 compile workers are unaffected; the mount only ever exists on the Linux worker.
- **\*\*Native path on Windows\*\*** ? the existing tests pass **\*\*as written\*\***, so they stay cross-platform rather than being marked Linux-only.

Considered and rejected the alternative (keep POSIX separators + make the test assert POSIX and skip on non-Linux): it would leave the helper Windows-broken for no benefit and drop local coverage.

## Verification (local Windows, worktree off `master`)

```
| Check | Result |  
|---|---|  
| `python run_all_tests.py --cov=backend --cov=training --cov-fail-under=70` | **1935 passed, 8 skipped, 0 failed** (exit 0) |  
| Baseline before fix | 1933 passed / **2 failed** ? the two target tests now pass |  
| Coverage | **84.28%** (floor 70; no regression vs. base) |  
| `ruff check` (0.15.16, CI version) | clean |  
| `mypy` (2.1.0, CI version) | clean |
```

Scope: 1 file, +7 / ?3 (`backend/storage/\_\_init\_\_.py`). No behavior change on Linux.

? Generated with [Claude Code](https://claude.com/claude-code)

### 62. user (2026-07-09T04:43:42.073Z)

File created successfully at: C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/d596f7ef-c2b1-4b96-b022-918ac3e40023/scratchpad/pr\_body.md (file state is current in your context ? no need to Read it back)

### 63. assistant (2026-07-09T04:43:48.966Z)

Now stage the explicit path, commit, re-verify only my commit is present, and push:

#### 64. user (2026-07-09T04:43:55.379Z)

```
[claude/gcsfuse-native-path d78a841] fix(storage): build GCSFuse fused path with OS-native
separators
 1 file changed, 7 insertions(+), 3 deletions(-)
===origin/master..HEAD (must be ONLY my commit)===
d78a841 fix(storage): build GCSFuse fused path with OS-native separators
===push===
remote: Create a pull request for 'claude/gcsfuse-native-path' on GitHub by visiting:
remote:      https://github.com/Khelsutra/badminton-highlight-indexer/pull/new/claude/gcsfuse-
native-path
remote:
branch 'claude/gcsfuse-native-path' set up to track 'origin/claude/gcsfuse-native-path'.
To https://github.com/Khelsutra/badminton-highlight-indexer.git
* [new branch]      claude/gcsfuse-native-path -> claude/gcsfuse-native-path
```

#### 65. assistant (2026-07-09T04:44:02.415Z)

Committed (`d78a841`), verified `origin/master..HEAD` contains **only my commit**, and pushed.  
Now creating the PR (non-draft = ready for review):

#### 66. user (2026-07-09T04:44:08.118Z)

<https://github.com/Khelsutra/badminton-highlight-indexer/pull/548>

#### 67. assistant (2026-07-09T04:44:21.146Z)

PR created: **<https://github.com/Khelsutra/badminton-highlight-indexer/pull/548>**. Let me confirm it's ready-for-review (non-draft), check for auto-requested reviewers (CODEOWNERS), and see if CI has started.

#### 68. user (2026-07-09T04:44:28.349Z)

```
===PR state===
{"additions":7,"baseRefName":"master","changedFiles":1,"deletions":3,"headRefName":"claude/gcsfu
se-native-path","isDraft":false,"mergeable":"MERGEABLE","number":548,"reviewRequests":[],"state"
:"OPEN","title":"fix(storage): OS-native GCSFuse fused path (fixes Windows-only #535 test
failures)"}
===CODEOWNERS?===
no CODEOWNERS
===checks===
Markdown link check (live docs) pass      8s      https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28994659178/job/86041629129
Built wheel smoke (install + launch)      pending 0      https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28994659178/job/86041629156
Full test suite (offline, mocked)         pending 0      https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28994659178/job/86041629141
Golden fixtures cross-OS (macos-latest) pending 0      https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28994659178/job/86041629153
Golden fixtures cross-OS (ubuntu-latest)  pending 0
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28994659178/job/86041629261
Mypy (lenient baseline) pending 0          https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28994659178/job/86041629113
Ruff lint                                  pending 0      https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28994659178/job/86041629107
Leak scan (PII / attribution guard)       pass      7s      https://github.com/Khelsutra/badminton-
highlight-indexer/actions/runs/28994659178/job/86041629117
Dependency license scan (no copyleft in runtime deps) pending 0
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28994659178/job/86041629149
Golden fixtures cross-OS (windows-latest) pending 0
https://github.com/Khelsutra/badminton-highlight-
indexer/actions/runs/28994659178/job/86041629139
```

Syntax sweep + import smoke (no GPU stack) pending 0  
<https://github.com/Khelsutra/badminton-highlight-indexer/actions/runs/28994659178/job/86041629136>

## 69. assistant (2026-07-09T04:45:18.114Z)

PR **#548** is open, **ready for review** (non-draft), `MERGEABLE`, base `master` ?  
`claude/gcsfuse-native-path`, 1 file +7/?3. CI has kicked off (Markdown-link-check and Leak-scan already green; the rest pending, including the Windows and Linux lanes).

Two things worth noting, then I'll confirm the CI story is consistent. There's no `CODEOWNERS` and you're the sole owner, so GitHub has no one to auto-request (it blocks self-review) ? the PR is simply open and ready. Let me verify *why* CI missed this ? I suspect the full suite runs Linux-only.

## 70. user (2026-07-09T04:45:23.461Z)

```
===workflow files===
.github/workflows/ci.yml
.github/workflows/nightly.yml
===runs-on for full suite + storage test coverage in CI===
origin/master:.github/workflows/ci.yml:22: runs-on: ubuntu-latest
origin/master:.github/workflows/ci.yml:41: runs-on: ubuntu-latest
origin/master:.github/workflows/ci.yml:56: runs-on: ubuntu-latest
origin/master:.github/workflows/ci.yml:72: runs-on: ubuntu-latest
origin/master:.github/workflows/ci.yml:91: runs-on: ubuntu-latest
origin/master:.github/workflows/ci.yml:103: full-suite:
origin/master:.github/workflows/ci.yml:105: runs-on: ubuntu-latest
origin/master:.github/workflows/ci.yml:145: - name: Run full suite (with coverage floor)
origin/master:.github/workflows/ci.yml:151: # the full suite x3 OSes. It replays the
Windows/Linux/macOS runner against the
origin/master:.github/workflows/ci.yml:159: os: [ubuntu-latest, windows-latest, macos-
latest]
origin/master:.github/workflows/ci.yml:160: runs-on: ${{ matrix.os }}
origin/master:.github/workflows/ci.yml:187: runs-on: ubuntu-latest
origin/master:.github/workflows/ci.yml:214: runs-on: ubuntu-latest
origin/master:.github/workflows/nightly.yml:13:# (lint · leak-scan · typecheck · import-smoke ·
full-suite · golden-fixtures ·
origin/master:.github/workflows/nightly.yml:46: runs-on: ubuntu-latest
origin/master:.github/workflows/nightly.yml:64: runs-on: ubuntu-latest
origin/master:.github/workflows/nightly.yml:79: runs-on: ubuntu-latest
origin/master:.github/workflows/nightly.yml:97: runs-on: ubuntu-latest
origin/master:.github/workflows/nightly.yml:109: # Full offline suite + coverage floor ?
mirrors ci.yml::full-suite. CPU-only
origin/master:.github/workflows/nightly.yml:113: full-suite:
origin/master:.github/workflows/nightly.yml:115: runs-on: ubuntu-latest
origin/master:.github/workflows/nightly.yml:140: - name: Run full suite (with coverage
floor)
origin/master:.github/workflows/nightly.yml:151: runs-on: ubuntu-latest
origin/master:.github/workflows/nightly.yml:172: runs-on: ubuntu-latest
origin/master:.github/workflows/nightly.yml:198: runs-on: ubuntu-latest
origin/master:.github/workflows/nightly.yml:236: needs: [lint, leak-scan, typecheck, import-
smoke, full-suite, golden-fixtures, license-scan, build-artifact]
origin/master:.github/workflows/nightly.yml:238: runs-on: ubuntu-latest
origin/master:.github/workflows/nightly.yml:255: 'One of ruff / leak-scan / mypy /
import-smoke / full-suite (cov>=70) /',
```

## 71. assistant (2026-07-09T04:45:34.671Z)

Let me read the cross-OS job definition precisely (the comment at line 151 is ambiguous) before asserting anything:

## 72. user (2026-07-09T04:45:39.033Z)

fatal: ambiguous argument 'origin\master;.github\workflows\ci.yml': unknown revision or path not in the working tree.

Use '--' to separate paths from revisions, like this:

'git <command> [<revision>...] -- [<file>...']'

### 73. assistant (2026-07-09T04:45:48.873Z)

Git Bash mangled the `rev:path` colon. The worktree's `ci.yml` is unchanged by my commit, so I'll just read it directly:

### 74. user (2026-07-09T04:45:53.652Z)

```
103     full-suite:
104         name: Full test suite (offline, mocked)
105         runs-on: ubuntu-latest
106         timeout-minutes: 30
107         steps:
108             - uses: actions/checkout@v6
109             - uses: actions/setup-python@v6
110               with:
111                 python-version: "3.13"
112                 cache: pip
113             - name: System libs for opencv-python
114               run: |
115                 # Drop the runner's pre-installed Microsoft apt repo (azure-cli): it
periodically
116                 # 403s on InRelease and aborts `apt-get update` under the `-e` shell, failing
the
117                 # build on an unrelated repo we don't use. libgl1/libglib2.0 are in Ubuntu's
archive.
118                 sudo rm -f /etc/apt/sources.list.d/*microsoft* /etc/apt/sources.list.d/*azure*
119                 sudo apt-get update
120                 sudo apt-get install -y libgl1 libglib2.0-0t64 || sudo apt-get install -y
libgl1 libglib2.0-0
121             - name: Install CPU-only torch (no CUDA wheels on a CPU runner)
122               # torch/torchvision are NOT in pyproject deps ? their CPU/CUDA wheels
123               # need an out-of-band index that PEP 621 can't express. Keep installing
124               # them separately here, before the editable install pulls the rest.
125               run: python -m pip install torch torchvision --index-url
https://download.pytorch.org/whl/cpu
126             - name: Install trackers (git-only dep ? PyPI wheel is broken)
127               # D13/P12: ByteTrack moved out of `supervision` (removed in 0.30.0) to the
128               # standalone `trackers` package. Its PyPI wheel ships metadata only (no
129               # source ? roboflow/trackers#474), so it MUST come from git. PEP 621 can't
130               # express a git URL, so it's not in pyproject.toml (same pattern as torch).
131               # Installed here so the two D13/P12 tests in tests/test_person_detector.py
132               # actually run in CI instead of skipping via pytest.importorskip.
133               run: python -m pip install -r requirements-trackers.txt
134             - name: Install package (editable) + dev tooling
135               # Editable PEP 621 install: runtime deps + the `dev` group
136               # (pytest, pytest-cov, httpx for the fastapi TestClient, ruff, mypy).
137               run: python -m pip install -e .[dev]
138               # obsreport (private, soft dep) is absent here: reporting tests skip via
139               # pytest.importorskip ? run_all_tests.py prints which. The coverage floor
140               # is therefore calibrated against THIS lane's number (local runs with
141               # obsreport installed measure a little higher).
142               # Floor calibrated 2026-06-11 against this lane's measured 72% (local
143               # with obsreport: 73%). Ratchet it UP as coverage grows; never down
144               # without an owner decision.
145             - name: Run full suite (with coverage floor)
146               run: python run_all_tests.py --cov=backend --cov=training --cov-report=term
--cov-fail-under=70
147
148         # Cross-OS determinism guard for the committed golden regression fixtures
149         # (docs/GOLDEN_REGRESSION_FIXTURES.md). The CONTROL replay path is pure-stdlib
```

```

150 # (only numpy, no cv2/torch), so this is a FAST, focused job ? it does NOT run
151 # the full suite x3 OSes. It replays the Windows/Linux/macOS runner against the
152 # committed (parse-compared) snapshots, catching any cross-OS float drift beyond
153 # the 1e-6 tolerance before it can flake a contributor's PR on another machine.
154 golden-crossos:
155     name: Golden fixtures cross-OS (${{ matrix.os }})
156     strategy:
157         fail-fast: false
158         matrix:
159             os: [ubuntu-latest, windows-latest, macos-latest]
160     runs-on: ${{ matrix.os }}
161     timeout-minutes: 15
162     steps:
163         - uses: actions/checkout@v6
164         - uses: actions/setup-python@v6
165           with:
166             python-version: "3.13"
167         - name: Install minimal deps (no GPU stack ? the CONTROL replay needs only numpy)
168           run: python -m pip install pytest fastapi uvicorn pydantic python-dotenv google-
genai numpy
169         - name: Golden-fixtures characterization (cross-OS determinism guard)
170           # Also runs the freeze-real / time-origin-reset / M2 refusal-gate tests
(synthetic /
171           # tmp_path only ? no GPU/network) so the reset metric-invariance (fix #2) and
the
172           # de-attribution paths get cross-OS coverage and surface any win/mac float
drift.
173           run: |
174             python -m backend.eval.golden_fixtures --check
175             python -m pytest tests/test_golden_fixtures.py
tests/test_golden_real_fixtures.py -q
176
177     # Commercial-clean licensing guardrail (MIT/Apache/BSD only ? PACKAGING_PLAN.md P0d).
Scans the
178     # RUNTIME dependency closure (`pip install .` ? no torch/dev) and fails on any
copyleft
179     # (GPL/AGPL/LGPL/SSPL). torch (BSD-3) is installed out-of-band and is not in this
closure. This
180     # automates a guardrail that was human-only; the report is printed for review either
way.
181     # D13/P12 carve-out: `trackers` (Apache-2.0; ByteTrack's new home) is also git-only
and thus
182     # absent from this pyproject closure ? UNGuarded here, same shape as torch. It's
Apache-2.0 and
183     # depends only on the still-declared `supervision` (MIT), so the closure is clean
today; if
184     # `trackers` ever pulls a new transitive dep, this gate won't catch it ? audit on
bump.
185     license-scan:
186         name: Dependency license scan (no copyleft in runtime deps)
187         runs-on: ubuntu-latest

```

## 75. assistant (2026-07-09T04:46:29.342Z)

Confirmed as fact: the `full-suite` CI job (which runs `test\_storage.py`) is **only** `ubuntu-latest` only\*\* (ci.yml:105). The **only** cross-OS matrix incl. `windows-latest` is `golden-crossos`, which runs **just golden fixtures** ? its own comment says it "does NOT run the full suite x3 OSes." So `test\_storage.py` never runs on Windows in CI, which is exactly why this path-separator bug stayed latent. That's a non-obvious, recurring gotcha (you develop on Windows; CI's full suite is Linux-only) worth saving to memory. Let me record it.

## 76. user (2026-07-09T04:46:30.040Z)

```

1 # Memory index
2

```

3 - ? **\*\*Start here (GPU-resident rollout):\*\*** [session-handoff-gpu-resident.md](session-handoff-gpu-resident.md) ? #506 implemented, PR #507 open; next: on-box parity gate + 15-clip golden gate.

4 - ? **\*\*LIVE deploy (2026-07-08):\*\*** [live-deployment-state.md](live-deployment-state.md) ? CP rev **\*\*00020-dtg\*\*** + worker on digest **\*\*7b62a8df\*\*** (master 304af7b; #525/#527/#528/#530). compile?L4+NVENC ON; validation stays on CP. Rollback ? 00019-96f / c6393d27. Release-process gaps ? **\*\*#532\*\***. Next: owner CUJ to measure compile?L4.

5 - ? **\*\*CLOSED 2026-07-08:\*\*** [followup-validation-latency-live-cuj.md](followup-validation-latency-live-cuj.md) ? live SPA CUJ filled both CF-gated cells on rev **\*\*00019\*\*** (validation wall **\*\*183 s\*\*** PASS + reel compiled; fresh clip, corroborates not re-measures Video-1). Docs **\*\*PR #522\*\***. Surfaced stitch bottleneck ? issues **\*\*#521\*\*** (compile?L4 + config placement) & **\*\*#520\*\*** (debug tracing).

6

7 - ? **\*\*Perf gate + nightly reorg (PR #547, 2026-07-09):\*\*** [perf-release-gate.md](perf-release-gate.md) ? local release perf gate (prod vs HEAD, per-stage GPU timing, gates) at `scripts/gates/perf/`; nightly CLIs consolidated into `scripts/gates/`. Finding: release **\*\*perf-neutral\*\*** (#531 within local noise). ? pre-existing Windows-only `test\_storage` #535 fails ? `task\_2e0a3b82`.

8

9 - [Actual repo location](actual-repo-location.md) ? primary working dir is an empty STUB; real code is under `kheutra-guru/badminton-highlight-indexer`.

10 - [Efficient timeouts, retries & Windows gcloud SSH](efficient-timeouts-and-retries.md) ? no silent retry loops; plain `gcloud compute ssh` (plink/pscp reject `-o`, mishandle `~`); diagnose-first, run long work detached.

11 - [GPU VM setup gotchas](gpu-vm-setup-gotchas.md) ? NVDEC libs, apt-lock-at-boot, torch2.x/pynvc; what #501 should encode.

12 - [LGTM only when merge-ready](lgtm-only-when-merge-ready.md) ? post LGTM only when genuinely satisfied a PR is merge-ready (verified), for all PRs.

13 - [GPU-resident modernization status](gpu-resident-modernization.md) ? #506 validated clean; the resize must match cv2.warpAffine via grid\_sample (not F.interpolate).

14 - [compute=local hosted guardrail](compute-local-hosted-guardrail.md) ? the "weights not found" trap fix: #511 + #169 + revendor #516 merged AND **\*\*DEPLOYED LIVE\*\*** (control-plane rev `00018-qvz`, image `sha256:8045f54e?`; rollback ? rev `00017-qbd`).

15 - [Validation shared decode pass](validation-shared-decode-pass.md) ? validators now share ONE decode pass (**\*\*PR #512 MERGED\*\***, squash 20862d7); pure grab()-through is SLOWER than seeks ? use forward-hybrid. **\*\*CLOSED 2026-07-07: DEPLOY\_REPORT + PR #519 MERGED (squash 71c88ef)\*\*** (real-L4 A/B 169?70 s = 2.43x; off-box 1.95?3.23x; zero decision drift). True-4K sub-second follow-up = issue #514.

16 - [PR #518 validation-result cache](pr518-validation-result-cache.md) ? reupload skips re-validation when config unchanged (video\_id-keyed cache, migration v10). **\*\*MERGED\*\*** 2026-07-07 (squash 699d85b; CI 11/11). ? v10 ? deploy worker+CP together.

17 - [PR #513 worker skip-revalidation](pr513-worker-skip-revalidation.md) ? worker skips redundant re-validation on cloud dispatch (fixes the blur-threshold CUJ). **\*\*MERGED\*\*** 2026-07-07 (squash 23c1868). **\*\*CLOSED live 2026-07-07: DEPLOY\_REPORT + PR #519 MERGED (squash 71c88ef)\*\*** ? dispatch-faithful worker A/B proved Video 1 FAIL?PASS / 13 rallies (validation 169 s+reject ? 0 s+PASS); reel + live CP wall stay owner-gated (CF Access 401). ? landed first on orchestration.py + test\_api\_cloud\_dispatch.py, so PR #511 rebased onto master (#512 was file-disjoint).

18 - [Cloud-run exec-status fast-fail](cloud-run-exec-status-fastfail.md) ? the #513 follow-up: `run\_infer` now probes the Executions API so a worker that crashes before the done-marker fails fast (rc 5) instead of a ~65-min hang. **\*\*PR #515 MERGED\*\*** 2026-07-07 (squash `ef41473`; CI 11/11 green). Adversarial review caught+fixed a proto-plus `seconds` blocker pre-merge.

19

## 77. assistant (2026-07-09T04:47:28.016Z)

MEMORY.md line 7 already flags this exact bug (from the PR #547 session) as `task\_2e0a3b82`. Now that I've fixed it in PR #548, I'll (a) record the non-obvious root cause as a new memory, and (b) update the index to mark the bug fixed.

## 78. assistant (2026-07-09T04:47:33.230Z)

---

name: ci-windows-coverage-gap



description: CI runs the full suite (incl. tests/test\_storage.py) on Linux only; the only Windows CI lane runs just golden fixtures ? Windows-only bugs escape CI.

metadata:  
type: project  
---

CI (.github/workflows/ci.yml) runs the **full suite** (run\_all\_tests.py, incl. tests/test\_storage.py) on **ubuntu-latest** only (the **full-suite** job). The **ONLY** cross-OS matrix that includes windows-latest/macos-latest is golden-crossos, and it runs **only** the golden-fixtures tests (tests/test\_golden\_fixtures.py + tests/test\_golden\_real\_fixtures.py) ? its own comment says it "does NOT run the full suite x3 OSes." nightly.yml's full-suite is also ubuntu-only.

**Why:** the owner develops on **Windows** but CI's full suite is **Linux-only**, so **Windows-only** bugs (path separators, os.sep, \ vs /, case-sensitivity) pass CI and only surface via local python run\_all\_tests.py on Windows. Exactly how the #535 GCSFuse path-join bug (mixed-separator fused path ...gcs\videos\clip.mp4) stayed latent until a local run ? fixed in **PR #548** (os.path.join(mount, \*key.split("/")) so the POSIX object key becomes OS-native). See [[actual-repo-location]].

**How to apply:** when touching filesystem-path code, use os.path.join/pathlib, never a hardcoded /\, and treat a green CI as NOT proving Windows-clean ? run the full suite locally on Windows before trusting portability. Note the coverage floor (70%) is calibrated against the CI lane WITHOUT the private obsreport dep (~72%); local runs WITH obsreport measure higher (~84%), so local vs CI coverage numbers legitimately differ.

#### 79. user (2026-07-09T04:47:33.321Z)

File created successfully at: C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\ci-windows-coverage-gap.md (file state is current in your context ? no need to Read it back)

#### 80. user (2026-07-09T04:47:35.772Z)

The file C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\MEMORY.md has been updated successfully. (file state is current in your context ? no need to Read it back)

#### 81. user (2026-07-09T04:47:38.629Z)

The file C:\Users\avidu\claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\MEMORY.md has been updated successfully. (file state is current in your context ? no need to Read it back)

#### 82. assistant (2026-07-09T04:48:01.230Z)

Memory recorded and index updated. Note: the background chip task\_2e0a3b82 (from the PR #547 session that flagged this) is now superseded by PR #548 ? it was spawned in another session so I can't dismiss it from here, but you can safely dismiss that chip. Let me grab a fresh CI snapshot for the final report.

#### 83. user (2026-07-09T04:48:08.174Z)

pass Built wheel smoke (install + launch)  
pass Dependency license scan (no copleft in runtime deps)  
pass Full test suite (offline, mocked)

```
pass Golden fixtures cross-OS (macos-latest)
pass Golden fixtures cross-OS (ubuntu-latest)
pass Golden fixtures cross-OS (windows-latest)
pass Leak scan (PII / attribution guard)
pass Markdown link check (live docs)
pass Mypy (lenient baseline)
pass Ruff lint
pass Syntax sweep + import smoke (no GPU stack)
---
pass: 11
```

#### 84. assistant (2026-07-09T04:48:27.131Z)

All **11** CI checks are green on PR #548 ? including the Linux **Full test suite** (with my fix), **Ruff**, **Mypy**, and **Golden fixtures cross-OS (windows/ubuntu/macos)**. Done.

#### ? PR #548 ? opened and ready for review

**<https://github.com/Khelsutra/badminton-highlight-indexer/pull/548>**

```
| | |
|---|---|
| State | OPEN, non-draft (ready for review), MERGEABLE |
| Base ? head | `master` ? `claude/gcsfuse-native-path` (cut from `origin/master` @ `3249451`) |
| Commit | `d78a841` ? `fix(storage): build GCSFuse fused path with OS-native separators` |
| Scope | 1 file, +7 / ?3 (`backend/storage/__init__.py`) |
| CI | 11/11 green (Full suite, Ruff, Mypy, cross-OS golden incl. windows-latest, wheel
smoke, license/leak scans) |
| Local Windows | full suite 1935 passed / 0 failed, coverage 84.28% (floor 70) |
```

**The fix** ? `\_resolve\_gcs\_fuse\_path` now splits the POSIX object key into segments so  
`os.path.join` builds an OS-native path (byte-identical on Linux, correct on Windows), instead  
of leaving the key's embedded `/` verbatim.

**Reviewers:** the repo has no `CODEOWNERS` and you're the sole owner, so GitHub auto-requested  
no one (it can't request self-review). The PR is simply open and ready ? assign whoever you'd  
like, or self-merge since CI is green.

#### Also handled

- **Memory:** recorded the non-obvious root-cause of *why* CI missed this ? [ci-windows-coverage-gap.md](../../.claude/projects/C--Users-avidu-Projects-badminton-highlight-indexer/memory/ci-windows-coverage-gap.md): CI's full suite (incl. `test\_storage.py`) is **Linux-only**; the Windows lane runs only golden fixtures, so Windows-only path bugs escape CI. Updated the MEMORY.md index (the `task\_2e0a3b82` line now shows **fixed in #548**).
- **Stale chip:** `task\_2e0a3b82` (from the PR #547 session that flagged this) is now addressed by #548 ? you can dismiss that chip; I couldn't from this session since it was spawned elsewhere.
- **Your shared checkout was untouched** ? all work happened in an isolated worktree off  
`origin/master`; the main checkout stays on `codex/issue-523-sd-reel-plan-clear` (and I left the  
stray untracked `SESSION\_HANDOFF (1).md` alone).

Want me to merge it (squash), or leave it for you to review and merge?